

Projeto A3

Disciplina: Teoria da Computação e Compiladores **Data:** 25/06/2026

Aluno(a): Julia Tizuey Martins Hosoy

RA: 12424217749

Link do Repositorio: https://github.com/Tizuey/minicalc_TrilhaDeAprendizado

Estudo de Caso - Tiny C Compiler (TCC)

1. Sobre o Compilador Tiny C Compiler (TCC)

O Tiny C Compiler, conhecido como TCC, é um compilador open source escrito em C que consegue compilar um subconjunto bem significativo da linguagem C ANSI. Apesar de ser pequeno (o código inteiro tem menos de 20 mil linhas), ele é totalmente funcional e consegue compilar programas reais, inclusive ele mesmo. O projeto foi criado e é mantido principalmente por Fabrice Bellard, e sua leveza o torna uma ferramenta interessante tanto para fins educacionais quanto para ambientes onde espaço ou recursos são limitados.

2. Por Que Escolhi o TCC?

Escolhi estudar o TCC porque ele é um compilador simples e consegue mostrar de forma clara as principais etapas da compilação. Diferente de compiladores como GCC e Clang, que são muito grandes e complexos, o TCC possui um código mais fácil de entender, sendo uma ótima opção para estudo. Além disso, ele é um compilador real, capaz de compilar programas em C, o que torna o aprendizado mais prático e facilita a compreensão dos conceitos de um compilador.

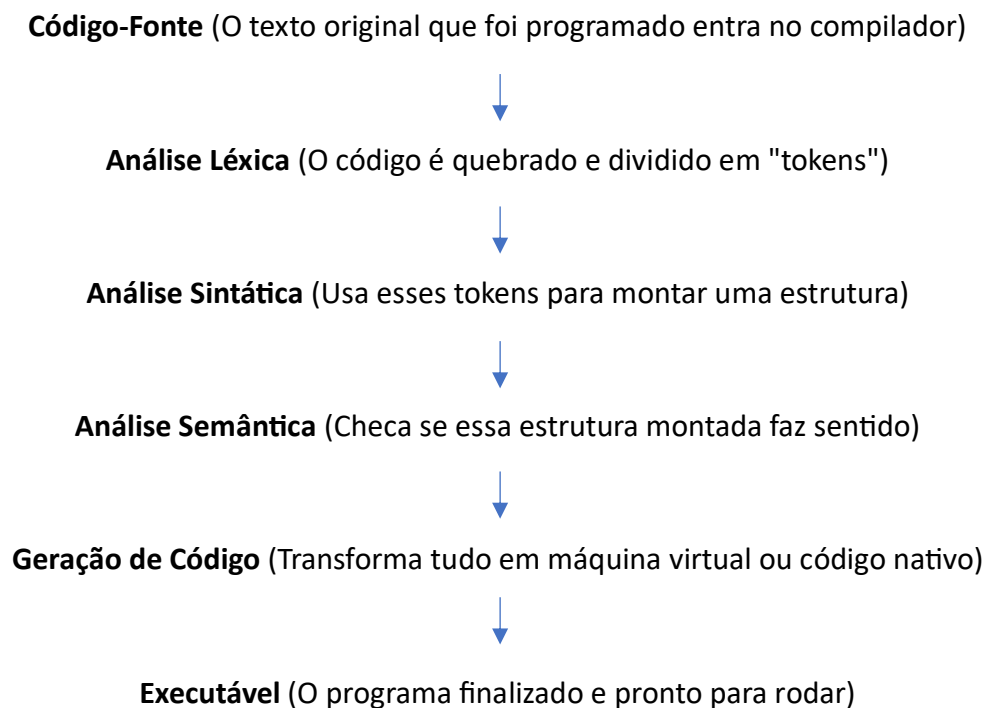
3. Objetivo e Contexto

O TCC foi desenvolvido com o objetivo de ser um compilador para a linguagem C que fosse pequeno, rápido e fácil de utilizar. Diferente de muitos compiladores, ele consegue gerar código executável de forma direta, sem depender de várias ferramentas externas. Além disso, seu código-fonte é relativamente simples, o que facilita o estudo de como um compilador funciona. Outra característica interessante é que ele também pode executar código C diretamente, sendo útil para testes e protótipos.

4. Como o TCC Funciona

O funcionamento do TCC segue as principais etapas encontradas em um compilador tradicional. Primeiro, ele recebe o código-fonte escrito em C. Em seguida, realiza a análise léxica para identificar os tokens, depois faz a análise sintática para verificar se o código segue a gramática da linguagem e monta sua estrutura. Após isso, ocorre a análise semântica, que verifica se as declarações, tipos e identificadores estão corretos. Por fim, é feita a geração do código executável.

Fluxo simplificado do TCC:



6. As Etapas de Compilação no TCC

A análise léxica é responsável por ler o código e separar seus elementos em tokens, como palavras-chave, identificadores, números e operadores.

Na análise sintática, esses tokens são organizados de acordo com a gramática da linguagem C. Nessa etapa é construída uma árvore sintática que representa a estrutura do programa.

Depois vem a análise semântica, que verifica se o programa faz sentido. São realizadas verificações como o uso correto de variáveis, compatibilidade entre tipos e existência das funções utilizadas.

Por último, ocorre a geração de código. Nessa fase, o TCC transforma a estrutura criada nas etapas anteriores em código executável para a arquitetura desejada.

Em relação às otimizações, o TCC possui poucas quando comparado a compiladores como GCC e Clang. Essa escolha foi feita para manter o compilador simples e com alta velocidade de compilação.

7. Características, Vantagens e Limitações

A principal vantagem do TCC é sua simplicidade. Por possuir um código menor, ele é mais fácil de estudar e compreender. Outro ponto positivo é sua velocidade de compilação e a possibilidade de executar código C diretamente em algumas situações.

Como limitação, o TCC não realiza otimizações avançadas. Por isso, o código gerado normalmente não apresenta o mesmo desempenho que programas compilados pelo GCC ou pelo Clang. Além disso, ele oferece suporte mais limitado a algumas extensões da linguagem C.

8. Onde é Usado

O TCC é bastante utilizado em estudos sobre compiladores por ser pequeno e fácil de entender. Também pode ser usado em ambientes com poucos recursos, em testes rápidos e em projetos que precisam compilar código durante a execução da aplicação.

9. Conclusão e Reflexão Crítica

O estudo do Tiny C Compiler ajudou a compreender melhor como funciona o processo de compilação. Mesmo sendo um compilador simples, ele implementa as principais etapas presentes em compiladores maiores, permitindo visualizar na prática conceitos estudados na disciplina.

Apesar de não possuir otimizações avançadas, o TCC atende muito bem ao seu propósito. Sua principal vantagem é mostrar de forma clara como um compilador é organizado, tornando o aprendizado mais acessível. Por isso, considero que ele foi uma boa escolha para entender o funcionamento interno de um compilador real.

Documentação Técnica: Arquitetura e Relatório Final do Compilador

1. Projeto da Arquitetura do Compilador Proposto

O MiniCalc foi pensado como um compilador que funciona em fases bem definidas, uma após a outra, sem voltar para refazer nada. Essa escolha deixa o código mais organizado e facilita encontrar problemas quando eles aparecem.

1.1 Pipeline de Dados

A ideia foi dividir o compilador em quatro partes que trabalham em sequência. A saída de uma parte vira entrada da próxima, bem linear mesmo.

Primeiro, você tem o arquivo de texto (.calc) com o código-fonte. Depois o Scanner, ou analisador léxico, lê esse texto e transforma em uma série de tokens. Pense em tokens como sendo as palavras da linguagem. O Parser pega esses tokens e verifica se eles seguem as regras gramaticais, construindo uma estrutura chamada Árvore Sintática Abstrata, ou AST. Por fim, o Code Generator percorre essa árvore e traduz para código Python, que é o que a gente realmente executa.

Essa separação é importante porque cada módulo tem uma responsabilidade clara. Se algo quebra, você sabe bem onde procurar.

1.2 A Gramática da Linguagem

Para conseguir fazer um parser que funciona bem, a gramática foi desenhada para ser LL(1). Isso é um jeito de dizer que o parser consegue entender a linguagem olhando apenas um token à frente, sem ambiguidade.

As regras ficaram assim:

Programa -> Comando*

Comando -> ID ASSIGN Expressao | PRINT Expressao

Expressao -> Termo ((PLUS | MINUS) Termo)*

Termo -> Fator ((MUL | DIV) Fator)*

Fator -> NUM | ID | LPAREN Expressao RPAREN

A forma como a gente organizou essas regras, colocando Expressao acima de Termo e Termo acima de Fator, faz a precedência matemática funcionar automaticamente.

Multiplicação acaba tendo prioridade sobre adição só por causa dessa hierarquia, o que é bem elegante.

2. Relatório Final: Análise, Geração de Código e Tecnologias

Aqui a gente consolida tudo que foi feito no projeto, explicando como o compilador funciona de verdade e as ferramentas que a gente usou.

2.1 Análise Léxica e Sintática (Front-end)

O analisador léxico funcionou bem. Ele consegue pegar o texto e transformar em tokens, e faz isso ignorando espaços em branco corretamente. A parte do parser também saiu robusta. Quando ele encontra um token que não deveria estar ali (tipo alguém escrevendo $x = + 5$), ele avisa que tem um erro sintático e para ali mesmo, antes de gerar uma árvore errada. Isso evita que erros da análise sintática causem problemas mais à frente na geração de código.

2.2 Como o Código é Gerado (Back-end)

Diferente dos compiladores tradicionais que geram código de máquina, o MiniCalc funciona de um jeito mais simples. Ele lê código fonte e escreve código Python. Por isso a gente chama de transpilador, não de compilador no sentido clássico.

O módulo que faz isso é recursivo. Basicamente, ele caminha pela árvore sintática de forma profunda e vai traduzindo cada nó. Quando encontra um número ou uma variável, simplesmente retorna o valor ou o nome. Quando encontra uma operação, avalia a parte esquerda, depois a direita, e junta tudo com o operador no meio, entre parênteses para deixar claro. Para coisas como atribuição, ele monta a estrutura `variavel = expressao`.

Essa abordagem funciona bem porque a árvore que o parser construiu tem exatamente a informação que a gente precisa para gerar código válido.

2.3 Ferramentas e Tecnologias

A gente decidiu não usar geradores de código automáticos como ANTLR ou YACC. O objetivo era realmente entender como compiladores funcionam por dentro, não só saber usar uma ferramenta.

Python 3 foi escolhido como linguagem. A tipagem dinâmica e as facilidades de string do Python deixam o desenvolvimento mais rápido e o código mais legível.

A biblioteca **re** (expressões regulares) foi usada na análise léxica para reconhecer os padrões da linguagem. Expressões regulares funcionam bem para isso.

unittest foi o framework usado para testes. Só para seguir um formato concreto de dev.

2.4 Conclusão

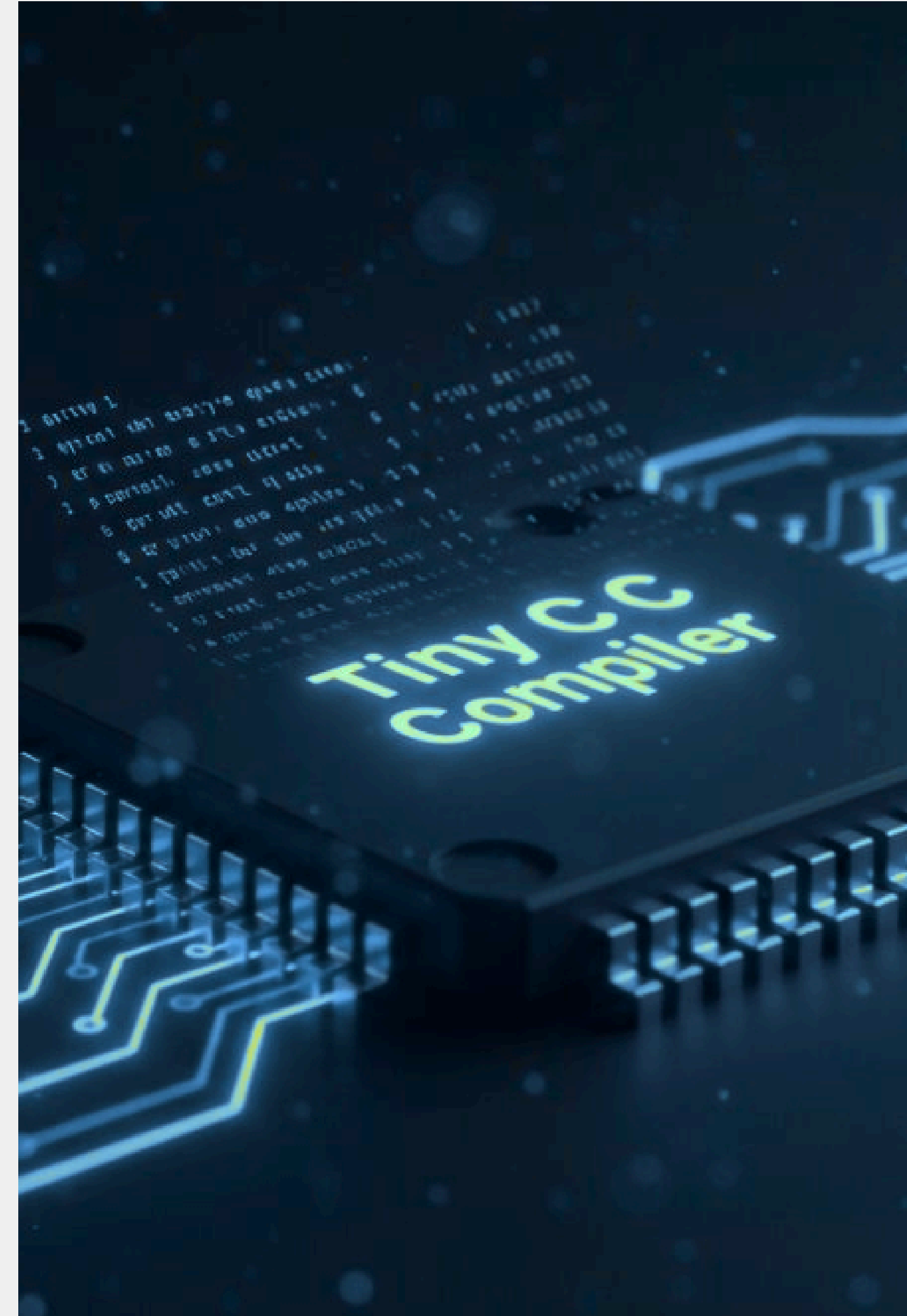
O projeto MiniCalc saiu do jeito que a gente planejou. Conseguimos levar a teoria que vimos em aula, aquela coisa de autômatos e gramáticas formais, para algo real que funciona. Deu para ver na prática por que compiladores são complexos e bem pensados. O código está estruturado de forma que no futuro a gente consegue adicionar coisas novas, como loops ou condicionais, sem quebrar o que já existe. Isso deixa bem claro o quanto aprendemos sobre compiladores nessa disciplina.



JULIA TIZUEY MARTINS HOSOY

Projeto A3: Protótipo de Compilador (MiniCalc)

Teoria da Computação e Compiladores



O Estudo de Caso

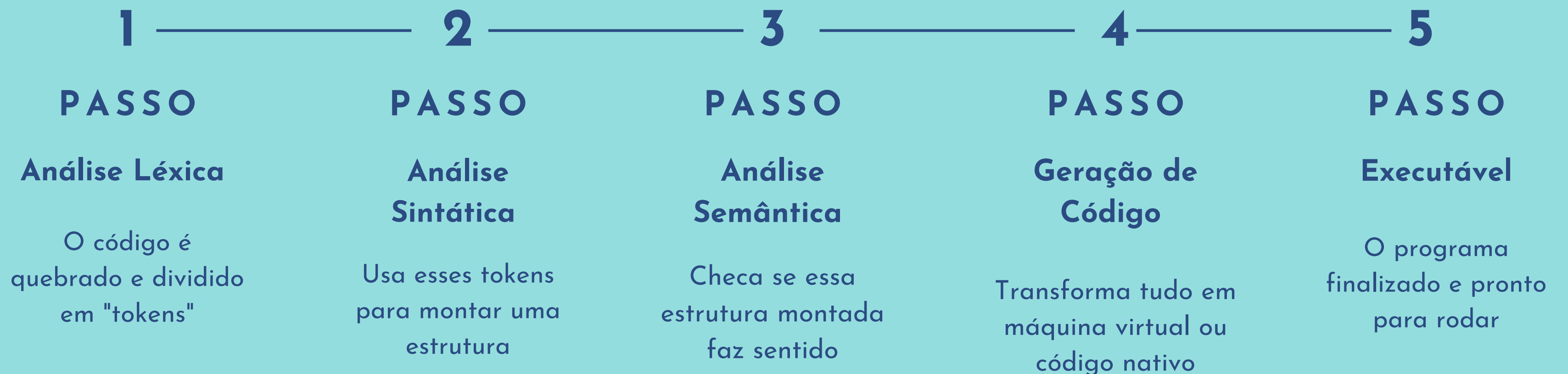
Compilador estudado: Tiny C Compiler (TCC).

- Compilador para a linguagem C.
- Pequeno, rápido e com código-fonte fácil de entender.
- Possui as principais etapas presentes em compiladores maiores.

O que foi aproveitado no projeto

- O MiniCalc foi desenvolvido seguindo a mesma ideia de dividir o processo em etapas: análise léxica, análise sintática, construção da AST e geração de código.

Fluxo simplificado do TCC





Arquitetura do MiniCalc

- O MiniCalc recebe um arquivo .calc e gera um código Python equivalente.

Fluxo do compilador:

- Código Fonte → Lexer → Parser → AST → Geração de Código Python
- Cada etapa possui uma responsabilidade específica, facilitando a organização do projeto e a identificação de erros.



Front-end: Lexer e Parser

Lexer

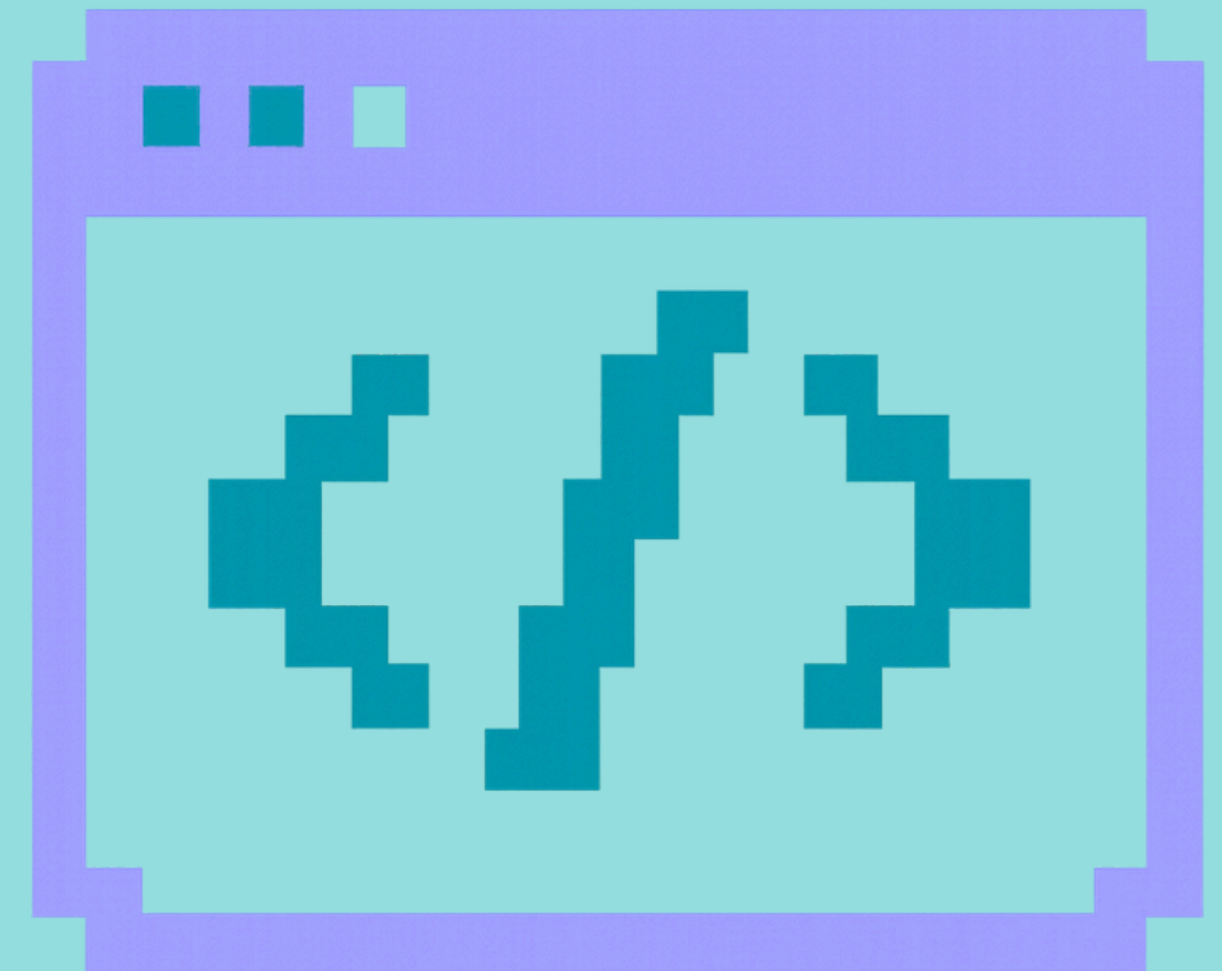
- O Lexer foi implementado utilizando Expressões Regulares da biblioteca re do Python.
- Ele é responsável por ler o código-fonte e identificar seus elementos, como números, operadores, identificadores e palavras-chave, transformando tudo em uma sequência de tokens.

Parser

- Cada regra da gramática possui um método correspondente, o que tornou o código mais organizado e fácil de relacionar com a teoria estudada em sala.
- Durante essa etapa também é construída a AST, que garante automaticamente a precedência correta das operações matemáticas.

Back-end: Geração de Código

- Após a construção da AST, o compilador percorre cada nó da árvore para gerar o código Python equivalente.
- Por exemplo, um nó de atribuição é convertido em uma instrução de atribuição do Python, enquanto um nó de operação matemática é convertido em uma expressão com os operadores correspondentes.
- Como a árvore já foi validada pelo Parser, a geração do código acontece de forma direta, produzindo um código Python válido e pronto para ser executado.



Demonstração Prática

(Exemplo de Execução)

CÓDIGO FONTE ORIGINAL DE ENTRADA (.CALC):

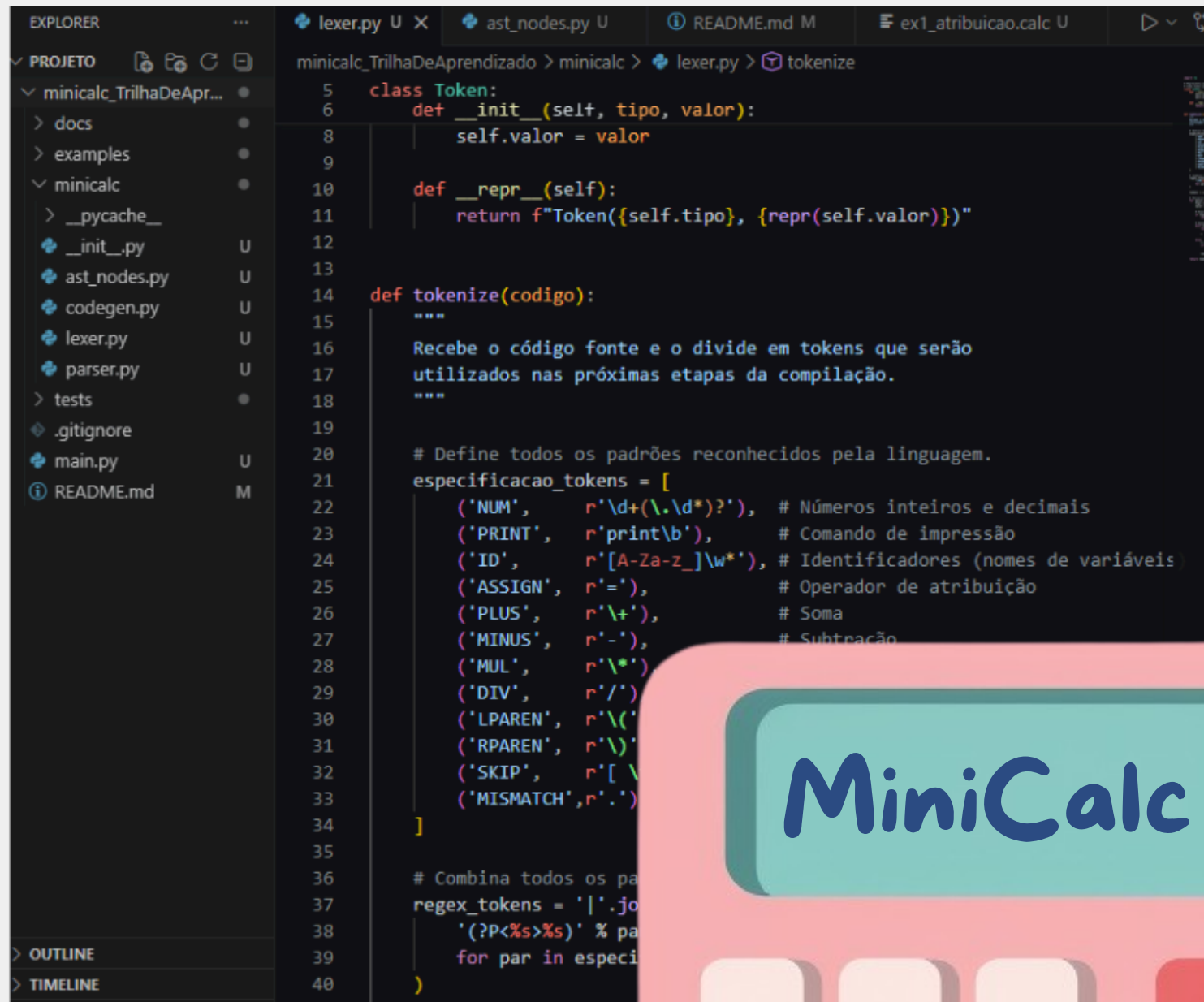
```
≡ ex3_print.calc U X
minicalc_TrilhaDeAprendizado > examples > ≡ ex3_print.calc
1  base = 5
2  altura = 10
3  area = base * altura / 2
4  print area
```

CÓDIGO FINAL GERADO COM SUCESSO (PYTHON):

```
base = 5
altura = 10
area = ((base * altura) / 2)
print(area)
```

PROCESSAMENTO INTERNO DO COMPILADOR:

- **Lexer:** Lê o texto e extrai os tokens em sequência: [ID: area], [ASSIGN: =], [ID: base], [MUL: *], [ID: altura], [DIV: /], [NUM: 2]...
- **Parser:** Monta a Árvore Sintática agrupando a multiplicação e a divisão de forma hierárquica ((base * altura) / 2).



```
5 class Token:
6     def __init__(self, tipo, valor):
7
8         self.valor = valor
9
10    def __repr__(self):
11        return f"Token({self.tipo}, {repr(self.valor)})"
12
13
14    def tokenize(codigo):
15        """
16        Recebe o código fonte e o divide em tokens que serão
17        utilizados nas próximas etapas da compilação.
18        """
19
20        # Define todos os padrões reconhecidos pela linguagem.
21        especificacao_tokens = [
22            ('NUM', r'\d+(\.\d*)?'), # Números inteiros e decimais
23            ('PRINT', r'print\b'), # Comando de impressão
24            ('ID', r'[A-Za-z_]\w*'), # Identificadores (nomes de variáveis)
25            ('ASSIGN', r'='), # Operador de atribuição
26            ('PLUS', r'\+'), # Soma
27            ('MINUS', r'-'), # Subtração
28            ('MUL', r'\*'),
29            ('DIV', r'/'),
30            ('LPAREN', r'\('),
31            ('RPAREN', r'\)'),
32            ('SKIP', r'[ \t\n]'),
33            ('MISMATCH', r'.'),
34        ]
35
36        # Combina todos os padrões em uma única expressão regular.
37        regex_tokens = '|'.join(f'({pattern})' for pattern, _ in especificacao_tokens)
38        regex_tokens = f'({regex_tokens})' % pattern
39        for par in especificacao_tokens:
40            pass
```



Conclusão Geral

- **Teoria na Prática:** A aplicação de autômatos e linguagens formais provou ser a fundação essencial e útil para a engenharia de software na prática.
- **Robustez:** O rigor sintático estabelecido no projeto previne que falhas lógicas drásticas cheguem ao usuário.
- **Escalabilidade:** O protótipo desenvolvido possui uma estrutura altamente escalável, permitindo a fácil inclusão de novas estruturas de controle (como if e while) em expansões futuras.

**Muito obrigada
professora pela
oportunidade.**

https://github.com/Tizuey/minicalc_TrilhaDeAprendizado

